

FavHost

Data Flow & Architecture Report

Generated: June 22, 2026 at 20:04

Covers: Calendar, Booking Details, Tasks, Listing & Registration pages

Table of Contents

1. Role System Overview
2. Visibility Engine - `get_visible_user_ids()` & `get_effective_user()`
3. Calendar Page (`calender.html`) - Data Flow
4. Booking Details Page (`payment_details.html`) - Data Flow
5. Task Pages - Data Flow
6. Listing Page - Data Flow
7. Registration Page - Data Flow
8. Cross-Page Data Alignment Analysis
9. Role-Based Data Differences Per Page
10. Summary

1. Role System Overview

Role	Description	Key Characteristics
Host	Property owner. Creates listings, owns all data.	Full profile; creates properties; owns bookings, tasks, enquiries.
Co-host	Granted access by a host via CoHost model.	No profile page; creations attributed to host via <code>get_effective_user()</code> ; can manage host's data.
Admin	Django superuser / staff.	Only relevant for Django admin panel; not used in frontend views.

[Note] Co-hosts are blocked from Profile and Edit Profile pages (`accounts/views.py:566, 734`).

2. Visibility Engine

2.1 `get_visible_user_ids(user)`

Core function in `accounts/utils.py:4`. Returns a list of user IDs whose data the given user can access.

Source	Logic	Who is included
1. Self	Always	The user's own ID
2. Hosts who added me	<code>CoHost.objects.filter(co_host=user)</code>	Hosts who added this user as a co-host
3. Co-hosts I added	<code>CoHost.objects.filter(host=user)</code>	Co-hosts this user added as a host

2.2 `get_effective_user(user)`

Returns the **host** if the current user is a co-host, otherwise returns the user themselves. All data creation (properties, tasks, etc.) is attributed to this effective user.

```
def get_effective_user(user):
    host = CoHost.objects.filter(co_host=user).select_related('host').first()
    if host:
        return host.host
    return user
```

2.3 Visibility Example

Host **Alice** adds **Bob** as a co-host. Alice owns **Property P1** and has **10 bookings**, **5 tasks**. Bob creates **3 more tasks** on P1.

Query	Bob's visible IDs	What Bob sees
<code>Property.objects.filter(created_by__in=...)</code>	{Bob, Alice}	P1 (Alice's) + any of Bob's own properties
<code>Booking.objects.filter(property__created_by__in=...)</code>	{Bob, Alice}	Alice's 10 bookings
<code>Task.objects.filter(created_by__in=...)</code> [Pattern B]	{Bob, Alice}	3 tasks Bob created (NOT Alice's 5)

Query	Bob's visible IDs	What Bob sees
Task.objects.filter(property__created_by__in=...) [Pattern A]	{Bob, Alice}	All 8 tasks (Alice's 5 + Bob's 3)

[Note] Core misalignment: Pattern A (by property owner) vs Pattern B (by creator) give different results for co-hosts.

3. Calendar Page (calender.html)

Page Info	
URL	/calendar-grid-view/
View	CalenderAPIView - bookaid/views.py:464
Template	templates/frontend/calender/calender.html
Auth	LoginRequiredMixin
Data delivery	Initial context (properties) + AJAX JSON (bookings, tasks, enquiries)

3.1 Initial Page Load - get_context_data()

Context Key	Model / Source	Filter	Description
rooms_data	Property	created_by__in=get_visible_user_ids(), status='Active'	List of property dicts for sidebar
bookings_data	None (empty list)	Populated via AJAX	Initial placeholder

3.2 AJAX - get_bookings_data()

Triggered with **x-requested-with: XMLHttpRequest**. Accepts **start_date** and **end_date** (YYYY-MM-DD). Returns JSON with 3 keys.

JSON Key	Model	Filter Details	Key Fields Returned
bookings	Booking	property__created_by__in, date range, .exclude(status='cancelled'), select_related(property, channel), prefetch_related(images, payments)	id, propertyId, start_date, end_date, guestName, guestAvatar, color, channellcon, channelName, bookingId, paymentDates, paymentsInfo, totalPrice, totalPayment, firstInstallment
blocked dates	PropertyBlockDate	property__created_by__in, date range, is_active=True	id, propertyId, start_date, end_date, color, type, reason
tasks	Task	property__created_by__in, date in range, completed=False, select_related(property)	id, propertyId, date, time, task_type, details, assigned_to, phone, completed, repeat, repeat_till, color
enquiries	Enquiry	property__created_by__in, date range, is_archive=False, select_related(property)	id, propertyId, start_date, end_date, guestName, guestAvatar, color, phone, email, adults, children, pets, notes

4. Booking Details Page (payment_details.html)

Page Info	
URL	/booking/payment-details/<uuid:pk>/
View	payment_details() - booking/views.py:521
Template	frontend/booking/payment_details.html (full) or payment_details_partial.html (?partial=true)
Auth	@login_required
Note	There is NO template named booking_detail.html. The details page is called 'Payment Details'.

4.1 Context Data

Context Key	Model / Source	Description
booking	Booking (pk lookup, property__created_by__in filter)	Full Booking model instance
nights	booking.total_nights	Computed nights from check-in/out
payment_schedule	booking.payments.prefetch_related('attachments').all()	List of dicts: id, name, amount, date, is_paid, notes, attachments
subtotal	booking.price_per_night * nights	Base price
total_price	subtotal + fees (or monthly if nights >= 30)	Final price
monthly_payment	booking.price_per_night * 30 if nights >= 30	Monthly rate
time_delta	Checkout == today	Boolean
booking_status	Computed: cancelled/past/current/upcoming	Status label

4.2 AJAX Endpoints on this Page

Endpoint	Method	Purpose
/booking/ajax/update-payment-status/	POST	Toggle payment.is_paid
/booking/ajax/add-payment-attachment/	POST FormData	Upload file to PaymentAttachment
/booking/ajax/delete-payment-attachment/	POST	Delete a PaymentAttachment
/booking/ajax/update-payment-notes/	POST	Save notes on a Payment
/booking/ajax/delete-guest-image/	POST	Delete a GuestImage
/booking/ajax/delete-guest-document/	POST	Delete a GuestDocument
/booking/cancel/<uuid:pk>/	POST	Cancel booking
/booking/render-email-form/	GET	Load email form modal
/booking/send-guest-email/	POST	Send email to guest
/booking/guest-receipt/<uuid:pk>/	GET	Load receipt modal

5. Task Pages

5.1 Task List Page

Page Info			
URL	/tasks/list/		
View	TaskListView - tasks/views.py:20		
Template	templates/frontend/tasks/list.html		
Auth	LoginRequiredMixin		

Context Key	Model / Source	Filter	Description
tasks	Task	created_by__in=get_visible_user_ids(), select_related('property'), status=pending done all, ?search= filter	Paginated task list (10/page) with is_overdue, days_until_due
all_tasks_count	Task	created_by__in same base	Total count
pending_tasks_count	Task	Same + completed=False	Pending count
done_tasks_count	Task	Same + completed=True	Done count
status_filter	GET param	Defaults to 'pending'	Current tab
search_query	GET param	Searches property__title, task_type, details, assigned_to	Search term

5.2 Task AJAX Endpoints

Endpoint	Method	Request Body	Response
/tasks/update-status/<pk>/	POST	{"completed": true/false}	{"success": bool, "pending_tasks_count": int, "done_tasks_count": int}
/tasks/delete/<pk>/	POST	{"delete_mode": "single" "all"}	{"success": bool, "message": str, counts}

5.3 Task Create / Edit

Action	URL	Template	Models Used
Create	/tasks/create/	templates/frontend/tasks/create.html	TaskForm, Property (active, visible for dropdown)
Edit	/tasks/edit/<pk>/	Same template (is_edit=True)	Task (pk lookup, created_by__in), Property

6. Listing Page

6.1 Public Listing Grid

Page Info			
URL (public)	/property/listing/<short_code>/		
URL (auth)	/property/listing/		
View	ListingPageView - property/views.py:728		
Template	templates/frontend/public_host_website/listing.html		
Pagination	6 per page		

Context Key	Model / Source	Filter	Description
properties	Property	short_code lookup (public) or created_by__in (auth), status='Active', optional check_in/check_out/guests filters	Paginated property cards
host	MyUser	short_code lookup or request.user	Host profile info (avatar, social, about)
check_in / check_out	GET params	None	Date filter values
guests_count	GET param	Defaults to '1'	Guest filter
num_nights	Computed	From check_in/check_out	Night count
paginator / page_obj	Django ListView built-in	None	Pagination

6.2 Listing Detail Page

Page Info		
URL	/property/listing-details/<short_code>/<slug>/	
View	ListingDetailView - property/views.py:818	
Template	templates/frontend/public_host_website/listing_details.html	

Context Key	Model / Source	Description
property	Property (slug lookup)	Full property instance
host	property.created_by (MyUser)	Host info: phone, whatsapp, profile_picture
images	property.images.all() (PropertyImage)	Gallery images
amenities	property.amenities.all().order_by('name')	Amenities list
short_code	URL kwarg	For links and sharing

7. Registration Page

Page Info	
URL	/accounts/register/
View	register_view - accounts/views.py:269
Template	templates/frontend/auth/login.html (combined login + signup)
Auth	No auth required (public)

7.1 Flow

Step	Action	Data Store	Context
1. GET	Redirects to /accounts/login/	None	login.html with signup panel hidden
2. POST form	Validate: first_name, last_name, email, password1, password2	Checks MyUser for duplicate email/username	show_signup=True, signup_data={...}
3. Valid POST	Generate 4-digit OTP, store in session	session['signup_otp'], session['signup_data']	show_otp_modal=True, otp_email=...
4. OTP entered	POST to /accounts/verify-otp/	Validate against session, create MyUser	Redirect to dashboard
5. Resend OTP	POST to /accounts/resend-otp/	Generate new OTP, update session	Re-render with OTP modal

7.2 Context Data (per step)

Context Key	Source	When Present
show_signup	GET ?mode=signup or POST error	Steps 2-3
signup_data	Submitted form values	Steps 2-3 (repopulate form)
show_otp_modal	True after valid POST	Step 3
otp_email	Submitted email	Step 3 (display in modal)
firebase_*	Django settings	Google sign-in config
form	Django LoginView default	Login form (login panel)

8. Cross-Page Data Alignment Analysis

8.1 The Two Filter Patterns

Pattern	Filter Field	Used By
A - By property owner	property__created_by__in=get_visible_user_ids()	Calendar (bookings, tasks, enquiries), Dashboard (bookings, revenue, enquiries), Booking List, Booking Details, Listing Page
B - By creator	created_by__in=get_visible_user_ids()	Task List page, Dashboard (task counts)

8.2 Identified Misalignments

Misalignment #1 - Task Filtering (Calendar vs Task List)

	Calendar (get_bookings_data)	Task List (TaskListView)
Filter field	property__created_by__in=get_visible_user_ids()	created_by__in=get_visible_user_ids()
File:Line	bookaid/views.py:583	tasks/views.py:43
Effect on co-host	Sees tasks on host's properties (by property owner)	Sees only tasks they personally created

[Note] A co-host can see the host's tasks on the Calendar but NOT on the Task List page. Conversely, tasks the co-host creates appear on the Task List but are queried differently on the Calendar.

Misalignment #2 - Calendar sidebar only shows active properties

get_context_data() filters Property.objects.filter(status='Active') for the left sidebar, but get_bookings_data() fetches bookings without a property status filter. Bookings on non-active properties render on the grid but the property is missing from the sidebar.

Misalignment #3 - Enquiries pattern same as tasks

Calendar fetches enquiries using property__created_by__in (Pattern A). If an Enquiry list page were added later using created_by__in (Pattern B), the same mismatch would occur.

9. Role-Based Data Differences Per Page

Page	Host (owner)	Co-host	Admin
Dashboard	All own + co-hosts' properties, bookings, revenue, tasks, enquiries	Host's properties, bookings, revenue, enquiries. Tasks: only own (Pattern B).	Django admin only
Calendar	All own + co-hosts' data via property ownership (Pattern A)	All property data from host. Tasks visible. Enquiries visible.	N/A
Task List	All tasks they created (Pattern B)	Only tasks they created. Host's tasks NOT visible.	N/A
Booking List	All bookings on own + co-hosts' properties (Pattern A)	All bookings on host's properties.	N/A
Booking Details	Full booking details with payment schedule	Same as host (by property ownership filter).	N/A
Listing (public)	Public view of own properties	N/A (co-host has no public listing page)	N/A
Registration	N/A (public page)	N/A	N/A
Profile	Full profile view and edit	BLOCKED - redirects to dashboard with error	Django admin

10. Summary

[*] Two inconsistent filter patterns exist - Pattern A (by property owner) is used for most data (bookings, payments, enquiries, calendar). Pattern B (by creator) is only used for tasks. This creates inconsistent visibility for co-hosts.

[*] Co-hosts have limited profile access - They cannot view or edit their own profile, and all their actions are attributed to the host via `get_effective_user()`.

[*] Calendar sidebar vs grid mismatch - The sidebar only lists active properties, but the grid can render bookings on inactive properties.

[*] Booking details lives under 'Payment Details' - There is no `booking_detail.html`; the details page is at `/booking/payment-details/<pk>/`.

[*] Registration uses OTP flow - Multi-step: form -> OTP email -> OTP verification -> user creation -> auto-login. No data is fetched for page load.

[*] All page data is scoped by `get_visible_user_ids()` - This ensures multi-user visibility while maintaining data isolation between unrelated hosts.